



## Working with Kafka

### A) Setup Kafka Server.

1) Download from <https://kafka.apache.org/downloads>.

**Scala 2.12 - kafka\_2.12-2.7.0.tgz (asc, sha512)**

2) Extract the Zip and you can see the folder :  
**kafka\_2.12-2.7.0**

3) Copy to E:\ Drive  
Now Kafka Home : **E:\kafka\_2.12-2.7.0**

\*E:\kafka\_2.12-2.7.0\bin\windows/\*.bat  
\*E:\kafka\_2.12-2.7.0\config

4) Create **myzookeeper\_data** folder under **E:\kafka\_2.12-2.7.0**

5) Update **zookeeper.properties** files

dataDir=E:\kafka\_2.12-2.7.0\myzookeeper\_data  
clientPort=2181

6) Create **mykafka\_logs** folder **E:\kafka\_2.12-2.7.0**

7) Update **server.properties**

**broker.id=5**  
num.network.threads=3  
num.io.threads=8  
socket.send.buffer.bytes=102400  
socket.receive.buffer.bytes=102400  
socket.request.max.bytes=104857600

**log.dirs=E:\kafka\_2.12-2.7.0\mykafka\_logs**

**num.partitions=1**  
**offsets.topic.num.partitions=1**  
**num.recovery.threads.per.data.dir=1**



```
offsets.topic.replication.factor=1
transaction.state.log.replication.factor=1
transaction.state.log.min.isr=1
min.insync.replicas=1
default.replication.factor=1
```

```
log.retention.hours=168
log.segment.bytes=1073741824
log.retention.check.interval.ms=300000
zookeeper.connect=localhost:2181
zookeeper.connection.timeout.ms=18000
group.initial.rebalance.delay.ms=0
```

8) Set the path for E:\kafka\_2.12-2.7.0\bin\windows

**B) Send and Receive the Messages with Console based Provider and Consumer.**

9) Starting Zookeeper server

```
zookeeper-server-start.bat E:\kafka_2.12-2.7.0\config\zookeeper.properties
```

10) Starting Kafka server

```
kafka-server-start.bat E:\kafka_2.12-2.7.0\config\server.properties
```

11) list the brokers running

```
zookeeper-shell.bat localhost:2181 ls /brokers/ids
```

12) Create the Kafka Topic

```
kafka-topics.bat --create --zookeeper localhost:2181 --replication-factor 1
--partitions 1 --topic myjltopic
```

```
kafka-topics.bat --create --zookeeper localhost:2181 --replication-factor 1
--partitions 1 --topic myhellotopic
```

13) Check the list of Topics available

```
kafka-topics.bat --list --zookeeper localhost:2181
```



14) Start the Kafka Producer

```
kafka-console-producer.bat --broker-list localhost:9092 --topic myjlctopic
```

15) Start the Kafka Consumer

```
kafka-console-consumer.bat --bootstrap-server localhost:9092  
--topic myjlctopic --from-beginning
```

**C) Develop Spring Boot Applications to Send or Produce Messages**

**MyKafkaProducer :Working Steps:**

1. Take the WorkSpace : **D:\Spring-MS\TestLabs\Kafka-Labs**
2. Copy BookPriceMS into **Kafka-Labs** folder
3. Open the Eclipse
4. Rename the Project as **MyKafkaProducer**
5. Update the pom.xml
6. Update application.properties
7. Rename package to **com.coursecube.kafkaproducer**
8. Rename Boot main class as **StartMyKafkaProducer** and update as follows

```
@SpringBootApplication  
public class StartMyKafkaProducer {  
    public static void main(String args[]) {  
        SpringApplication.run(StartMyKafkaProducer.class, args);  
    }  
}
```

9. Write the Bean Configuration class with the required Beans

```
***ProducerFactory  
***KafkaTemplate
```

10. Write the TestService to send the Message to Kafka Server

```
@Autowired  
KafkaTemplate<String, String> stringKafkaTemplate;  
  
public void sendMessage(String message) {  
    stringKafkaTemplate.send("myjlctopic", message);  
}
```



11. Write the TestController

```
@Autowired
TestService testService;

@GetMapping("/testMsg")
public String testMessage() {
    testService.sendMessage("Hello Guys - This is message 1");
    return "String Message Sent to Kafka Server ";
}
```

12. Start the Boot Application

13. Hit URL : <http://localhost:12345/testMsg>

#### **D) Develop Spring Boot Applications to Receive or Listen Messages**

##### **MyKafkaConsumer: Working Steps:**

1. Copy **MyKafkaProducer** as **MyKafkaConsumer**
2. Update the pom.xml
3. Update application.properties
4. Rename package to **com.coursecube.kafkaconsumer**
5. Rename Boot main class as **StartMyKafkaConsumer** and update as follows

```
@SpringBootApplication
public class StartMyKafkaConsumer {
    public static void main(String args[]) {
        SpringApplication.run(StartMyKafkaConsumer.class, args);
    }
}
```

6. Write the Bean Configuration class (MyAppConfig) with the required Beans

```
***ConsumerFactory
***ConcurrentKafkaListenerContainerFactory
```

7. Write the TestService to receive the Message from Kafka Server

```
@KafkaListener(topics = "myjlcopic", groupId = "myjlc_group")
public void receiveMessage(String msg) {
    System.out.println(msg);
}
```

8. Start the Boot Application



## MyKafkaProducer – Files Required

|                              |                           |
|------------------------------|---------------------------|
| 1. StartMyKafkaProducer.java | 2. TestController.java    |
| 3. TestService.java          | 4. MyAppConfig.java       |
| 5. Order.java                | 6. application.properties |
| 7. pom.xml                   |                           |

### 1. StartMyKafkaProducer.java

```
package com.coursecube.kafkaproducer;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@SpringBootApplication
public class StartMyKafkaProducer {
    public static void main(String args[]) {
        SpringApplication.run(StartMyKafkaProducer.class, args);
    }
}
```

### 2. TestController.java

```
package com.coursecube.kafkaproducer;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import com.coursecube.kafka.Order;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@RestController
public class TestController {

    @Autowired
    TestService testService;
```



```
@Value(value = "${kafka.bootstrapAddress}")
private String bootstrapAddress;

@GetMapping("/testMsg")
public String testMessage() {
    System.out.println("---TestController---testMessage()---");
    testService.sendMessage("Hello Guys - This is message 1");
    testService.sendMessage("Hello Guys - This is message 2");
    return "String Message Sent to Kafka Server @ " + bootstrapAddress ;
}

@GetMapping("/testOrder")
public String placeOrder() {
    System.out.println("---TestController---placeOrder()---");
    Order myorder = new Order(5001, "19-Dec-2020", "C-101",5, 500,"New");
    testService.sendOrder(myorder);
    testService.sendOrder(myorder);
    return "Order Sent to Kafka Server @ " + bootstrapAddress ;
}
}
```

### 3. TestService.java

```
package com.coursecube.kafkaproducer;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;

import com.coursecube.kafka.Order;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@Service
public class TestService {

    @Autowired
    KafkaTemplate<String, String> stringKafkaTemplate; //1

    @Autowired
    KafkaTemplate<String, Order> orderKafkaTemplate; //1
}
```



```
private static final String MyHelloTopic="myhellotopic"; //2
private static final String MyOrderTopic="myordertopic"; //2

public void sendMessage(String message) {
    System.out.println("TestService -sendMessage()");
    stringKafkaTemplate.send(MyHelloTopic, message); //3
}

public void sendOrder(Order myorder) {
    System.out.println("TestService -sendOrder()");
    orderKafkaTemplate.send(MyOrderTopic, myorder); //3
}
}
```

#### 4. MyAppConfig.java

```
package com.coursecube.kafkaproducer;

import java.util.HashMap;
import java.util.Map;

import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.common.serialization.StringSerializer;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.kafka.core.DefaultKafkaProducerFactory;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.core.ProducerFactory;
import org.springframework.kafka.support.serializer.JsonSerializer;

import com.coursecube.kafka.Order;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@SpringBootApplication
public class MyAppConfig {

    @Value(value = "${kafka.bootstrapAddress}")
    private String bootstrapAddress;
```





@Bean

**public ProducerFactory<String, String> stringProducerFactory() {**

Map<String, Object> props = new HashMap<>();

props.put(ProducerConfig.BOOTSTRAP\_SERVERS\_CONFIG, bootstrapAddress);  
props.put(ProducerConfig.KEY\_SERIALIZER\_CLASS\_CONFIG, StringSerializer.class);  
props.put(ProducerConfig.VALUE\_SERIALIZER\_CLASS\_CONFIG, StringSerializer.class);

**return new DefaultKafkaProducerFactory<>(props);**  
**}**

@Bean

**public ProducerFactory<String, Order> orderProducerFactory() {**

Map<String, Object> props = new HashMap<>();

props.put(ProducerConfig.BOOTSTRAP\_SERVERS\_CONFIG, bootstrapAddress);  
props.put(ProducerConfig.KEY\_SERIALIZER\_CLASS\_CONFIG, StringSerializer.class);  
props.put(ProducerConfig.VALUE\_SERIALIZER\_CLASS\_CONFIG, JsonSerializer.class);

**return new DefaultKafkaProducerFactory<>(props);**  
**}**

@Bean(name="stringKafkaTemplate")

public KafkaTemplate<String, String> stringKafkaTemplate( ) {

**return new KafkaTemplate<>(stringProducerFactory());**

**}**

@Bean(name="orderKafkaTemplate")

public KafkaTemplate<String, Order> orderKafkaTemplate( ) {

**return new KafkaTemplate<>(orderProducerFactory());**

**}**

**}**





## 5. Order.java

```
package com.coursecube.kafka;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
public class Order {
    private Integer orderId;
    private String orderDate;
    private String userId;
    private int totalQty;
    private double totalCost;
    private String status;

    //Constructors
    //Setters and Getters
    //toString()
}
```

## 6. application.properties

```
spring.application.name=MyKafkaProducer

server.port=12345

kafka.bootstrapAddress=localhost:9092
```

## 7. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.coursecube.kafkaproducer</groupId>
<artifactId>MyKafkaProducer</artifactId>
<version>1.2</version>
<packaging>jar</packaging>

<name>MyKafkaProducer</name>
<url>https://www.coursecube.com</url>
<description>This is MyKafkaProducer developed by Srinivas Dande</description>
```



```
<properties>
<java.version>1.8</java.version>
<maven-jar-plugin.version>3.1.1</maven-jar-plugin.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
<relativePath/>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.kafka</groupId>
<artifactId>spring-kafka</artifactId>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```



## MyKafkaConsumer – Files Required

|                              |                     |
|------------------------------|---------------------|
| 1. StartMyKafkaConsumer.java | 2. TestService.java |
| 3. MyAppConfig.java          | 4. Order.java       |
| 5. application.properties    | 6. pom.xml          |

### 1. StartMyKafkaConsumer.java

```
package com.coursecube.kafkaconsumer;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@SpringBootApplication
public class StartMyKafkaConsumer {
    public static void main(String args[]) {
        SpringApplication.run(StartMyKafkaConsumer.class, args);
    }
}
```

### 2. TestService.java

```
package com.coursecube.kafkaconsumer;

import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Service;

import com.coursecube.kafka.Order;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@Service
public class TestService {

    @KafkaListener(topics = "myhellotopic",groupId = "myjlc_group",
        containerFactory = "stringKafkaListenerContainerFactory")
    public void receiveMessage(String msg) {
        System.out.println("TestService - receiveMessage()");
        System.out.println(msg);
    }
}
```



```
@KafkaListener(topics = "myordertopic",groupId = "myorder_group",
containerFactory = "orderKafkaListenerContainerFactory")
public void receiveOrder(Order myorder) {
    System.out.println("TestService - receiveOrder()");
    System.out.println(myorder);
}
}
```

### 3. MyAppConfig.java

```
package com.coursecube.kafkaconsumer;

import java.util.HashMap;
import java.util.Map;

import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.common.serialization.StringDeserializer;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.kafka.annotation.EnableKafka;
import org.springframework.kafka.config.ConcurrentKafkaListenerContainerFactory;
import org.springframework.kafka.core.ConsumerFactory;
import org.springframework.kafka.core.DefaultKafkaConsumerFactory;
import org.springframework.kafka.support.serializer.JsonDeserializer;

import com.coursecube.kafka.Order;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */

@SpringBootApplication
@EnableKafka
public class MyAppConfig {

    @Value(value = "${kafka.bootstrapAddress}")
    private String bootstrapAddress;
```



@Bean

```
public ConsumerFactory<String, String> stringConsumerFactory() {  
System.out.println("stringConsumerFactory");  
Map<String, Object> props = new HashMap<>();  
  
props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapAddress);  
props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);  
props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);  
props.put(ConsumerConfig.GROUP_ID_CONFIG, "myjlc_group");  
props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");  
  
return new DefaultKafkaConsumerFactory<>(props);  
}
```

@Bean(name="stringKafkaListenerContainerFactory")

```
public ConcurrentKafkaListenerContainerFactory<String, String>  
stringKafkaListenerContainerFactory() {  
System.out.println("stringKafkaListenerContainerFactory");  
  
ConcurrentKafkaListenerContainerFactory<String, String> factory = new  
ConcurrentKafkaListenerContainerFactory<>();  
  
factory.setConsumerFactory(stringConsumerFactory());  
  
return factory;  
}
```

@Bean

```
public ConsumerFactory<String, Order> orderConsumerFactory() {  
System.out.println("orderConsumerFactory");  
Map<String, Object> props = new HashMap<>();  
  
props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapAddress);  
props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);  
props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, JsonSerializer.class);  
props.put(ConsumerConfig.GROUP_ID_CONFIG, "myorder_group");  
props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");  
props.put(JsonDeserializer.TRUSTED_PACKAGES, "com.coursecube.kafka");  
  
return new DefaultKafkaConsumerFactory<String, Order>(props);  
}
```



```
@Bean(name = "orderKafkaListenerContainerFactory")
public ConcurrentKafkaListenerContainerFactory<String, Order>
orderKafkaListenerContainerFactory() {
    System.out.println("orderKafkaListenerContainerFactory");

    ConcurrentKafkaListenerContainerFactory<String, Order> factory = new
    ConcurrentKafkaListenerContainerFactory<>();

    factory.setConsumerFactory(orderConsumerFactory());

    return factory;
}
}
```

#### **4. Order.java**

```
package com.coursecube.kafka;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
public class Order {
    private Integer orderId;
    private String orderDate;
    private String userId;
    private int totalQty;
    private double totalCost;
    private String status;

    //Constructors
    //Setters and Getters
    //toString()
}
```

#### **5. application.properties**

```
spring.application.name=MyKafkaConsumer

server.port=54321

kafka.bootstrapAddress=localhost:9092
```



## 7. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.coursecube.kafkaconsumer</groupId>
<artifactId>MyKafkaConsumer</artifactId>
<version>1.2</version>
<packaging>jar</packaging>
<name>MyKafkaConsumer</name>
<url>https://www.coursecube.com</url>
<description>This is MyKafkaConsumer developed by Srinivas Dande</description>

<properties>
<java.version>1.8</java.version>
<maven-jar-plugin.version>3.1.1</maven-jar-plugin.version>
</properties>
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
<relativePath/>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.kafka</groupId>
<artifactId>spring-kafka</artifactId>
</dependency>
</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```